STOCHASTIC MODELING

MODULE 2 | LESSON 4

FOURIER-PRICING UNDER LEWIS (2001)

Reading Time	45 minutes
Prior Knowledge	Fourier Transform, Characteristic Function (CF), Derivative Pricing
Keywords	Fourier Transform, Inverse Fourier Transform, Lewis (2001)

In this lesson we will implement the Lewis (2001) method for option pricing. Similar to the Carr-Madan method we saw in the previous lesson, Lewis also leverages Fourier transforms, but works a bit differently.

As in the previous lesson, we will benchmark against the analytical Black-Scholes solution to check how things converge.

Let's start by importing necessary libraries:

In [1]:
```
import numpy as np
from scipy import stats
from scipy.integrate import quad
```

For further comparison across methods, let's price the European Call option from the previous lesson with same characteristics:

In [2]:
```
S0 = 100
K = 100
T = 1
r = 0.05
sigma = 0.2
```

1. Analytical Solution to BS Model

We will start with the already known analytical solution to the BS model:

In [3]:
```
def BS_analytic_call(S0, K, T, r, sigma):
    """This function will provide the closed-form solution
    to the European Call Option price based on BS formula
    """

    d1 = (np.log(S0 / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T
    d2 = (np.log(S0 / K) + (r - 0.5 * sigma**2) * T) / (sigma * np.sqrt(T
```

```
    bs_call = S0 * stats.norm.cdf(d1, 0.0, 1.0) - K * np.exp(-r * T) * st
        d2, 0.0, 1.0
    )

    return bs_call
```

In [4]:
```
print(
    " BS Analytical Call Option price will be $",
    BS_analytic_call(S0, K, T, r, sigma).round(4),
)
```

```
BS Analytical Call Option price will be $ 10.4506
```

2. Lewis(2001) for the Black-Scholes model

Next, we will deal with Lewis's approach to obtaining a semi-analytical solution to the price of the option via Fourier methods. As you know already, this method requires a bunch of things to know ex-ante:

2.1 Black-Scholes Characteristic Function

Fourier pricing methods require that we know the characteristic function of the process S_t (or some form of it like $s_t = log S_t$). In the case of a BS model (without dividends), the characteristic function is given by:

$$ \varphi^{BS} (u, T) = e^{((r-\frac{\sigma^2}{2})iu - \frac{\sigma^2}{2}u^2)T} $$

Let's implement a function for it as we did in the FFT method:

In [5]:
```
def BS_characteristic_func(v, x0, T, r, sigma):
    """Computes general Black-Scholes model characteristic function
    to be used in Fourier pricing methods like Lewis (2001) and Carr-Mada
    """

    cf_value = np.exp(
        ((x0 / T + r - 0.5 * sigma**2) * 1j * v - 0.5 * sigma**2 * v**2)
    )

    return cf_value
```

2.2 Integral Value in Lewis

We also need to get a value for the integral in Lewis:

$$ C_0 = S_0 - \frac{\sqrt{S_0 K} e^{-rT}}{\pi} \int_{0}^{\infty} \mathbf{Re}[e^{izk} \varphi(z-i/2)] \frac{dz}{z^2+1/4} $$

We will next compute a function for what's inside the integral:

In [6]:
```
def BS_integral(u, S0, K, T, r, sigma):
    """Expression for the integral in Lewis (2001)"""

    cf_value = BS_characteristic_func(u - 1j * 0.5, 0.0, T, r, sigma)

    int_value = (
        1 / (u**2 + 0.25) * (np.exp(1j * u * (np.log(S0 / K))) * cf_value
    )

    return int_value
```

2.3 Calculating the Value of the Integral and Putting It All Together

Now we can put everything together. Note, that we will need to numerically compute the value of the aforementioned integral. There are different methods to do that. For now, we will use the quadrature method (*quad*) included in the scipy package: https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.quad.html

In [7]:
```
def BS_call_Lewis(S0, K, T, r, sigma):
    """European Call option price in BS under Lewis (2001)"""

    int_value = quad(lambda u: BS_integral(u, S0, K, T, r, sigma), 0, 100

    call_value = max(0, S0 - np.exp(-r * T) * (np.sqrt(S0 * K)) / np.pi *

    return call_value
```

In [8]:
```
print(
    " Fourier Call Option price under Lewis (2001) is $",
    BS_call_Lewis(S0, K, T, r, sigma).round(4),
)
```

```
Fourier Call Option price under Lewis (2001) is $ 10.4506
```

3. Execution Time

As in previous lesson, the two methods yield the same European option price. Let's now explore the differences in execution time related to each.

In [9]:
```
import datetime
```

In [10]:
```
# BS Closed-form
begin = datetime.datetime.now()
price = BS_analytic_call(S0, K, T, r, sigma)
print(
    "BS closed-from price is $",
    price.round(4),
    ".Code took ",
    datetime.datetime.now() - begin,
)
```

```
BS closed-from price is $ 10.4506 .Code took  0:00:00.000310
```

In [11]:
```
# BS via FFT
begin = datetime.datetime.now()
price = BS_call_Lewis(S0, K, T, r, sigma)
print(
    "BS closed-from price is $",
    price.round(4),
    ".Code took ",
    datetime.datetime.now() - begin,
)
```

```
BS closed-from price is $ 10.4506 .Code took  0:00:00.000868
```

As was the case with FFT, the analytical approach is typically faster. The comparison between Carr-Madan and Lewis methods is not always clear although, given the alledgly small differences in computational times, Lewis is usually the preferred method.

In [12]:
```
from IPython.display import VimeoVideo

# Bigger video
VimeoVideo("1116720265", h="3298dbabb7", width=700, height=450)
```

Out[12]:

4. Conclusion

Well done! Now you know how to price vanilla options leveraging Fourier transforms under both Carr-Madan (FFT) and Lewis methods. These are going to play a central role in our next endeavour: model calibration.

See you in the next module!

References

- Hilpisch, Yves. *Derivatives Analytics with Python: Data Analysis, Models, Simulation, Calibration and Hedging*. John Wiley & Sons, 2015.
- Lewis, Alan L. "A Simple Option Formula for General Jump-Diffusion and Other Exponential Lévy Processes." 2001.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.